

Classificação KNN na base Iris: implementação do zero vs. Scikit-learn

Autores: Thales Maia Mendonça Negrão

Disciplina: GCC 128 - Inteligência Artificial

Algoritmo

K-Nearest Neighbors (KNN)

Dataset

Iris — 150 amostras, 3 classes

Abordagens

Implementação manual vs. Scikit-learn

Valores de K

$K \in \{1, 3, 5, 7\}$

Objetivo

O que este trabalho propõe

Implementar o algoritmo KNN de forma manual e comparar seu desempenho com a implementação disponível na biblioteca Scikit-learn, avaliando ambas as abordagens sob as mesmas condições experimentais.

Critérios de avaliação

1 Variação de K

Avaliação para $K = \{1, 3, 5, 7\}$, cobrindo cenários de baixa e alta suavização de fronteira de decisão.

2 Métricas de desempenho

Uso de **matriz de confusão** + métricas quantitativas: **precisão**, **revocação** e **acurácia** (média macro).

Como o KNN funciona



Calcular
distância

Selecionar K
vizinhos

Votação
majoritária

Para cada ponto de teste

- Calcula a distância para **todos** os pontos de treino
- Seleciona os **K vizinhos mais próximos**
- Faz **votação majoritária** para prever a classe

Quanto maior o K

- Tende a **suavizar ruído** nos dados
- Pode **reduzir sensibilidade** a variações locais
- Risco de perder fronteiras de decisão "locais"

Implementação do KNN do zero (hardcore)



Leitura dos dados

Lê `Iris.csv`, separa `X` (atributos numéricos) e `y` (classe alvo).



Cálculo de distância

Função `euclidean_dist(a, b)` usa distância Euclidiana entre duas amostras.



Métricas e matriz de confusão

`confusion_matrix_manual(...)` e `precision_recall_accuracy(...)` calculam precisão e revocação por classe (média macro). `plot_confusion(...)` plota a matriz para cada `K`.



Divisão treino/teste

`def train_test_split_simple(X, y, test_ratio=0.2, seed=42)` - divide os dados em treino e teste (80/20) e treina o modelo classificando dados.



Predição do KNN

Função `knn_predict_one(...)`: calcula distâncias, ordena, seleciona `K` vizinhos, faz votação e resolve desempate quando necessário.



Loop principal

Para cada $K \in \{1, 3, 5, 7\}$: gera predições, calcula métricas e plota matriz de confusão.

Implementação do KNN com Scikit-learn

→ Leitura do Iris.csv

Mesma estrutura: separação de `X` (atributos) e `y` (classe alvo).

→ Divisão treino/teste

Utiliza `train_test_split(x, y, test_ratio=0.2, seed = 42)` para garantir proporção de classes em ambos os conjuntos.

→ Modelo

`KNeighborsClassifier(n_neighbors=k, metric="euclidean")` — mesma métrica de distância da implementação manual.

→ Treino

`model.fit(X_train, y_tr)` — ajuste do modelo ao conjunto de treino.

→ Predição

`model.predict(X_test)` — geração das classes previstas para o conjunto de teste.

→ Avaliação e plots

`confusion_matrix(...)`, `precision_score(..., average="macro")`, `recall_score(..., average="macro")`, `accuracy_score(...)` e `plot_confusion(...)`.

📄 A implementação sklearn replica exatamente as mesmas condições da versão manual: mesma métrica euclidiana, mesmos valores de K e mesmo esquema de divisão treino/teste.

Resultados: o que aconteceu ao variar K

Comparação de métricas — Hardcore vs. Scikit-learn (preencha com os resultados obtidos)

K = 1 (pequeno)

Pode aumentar acurácia em treino, mas fica mais **sensível a ruído** — overfitting local.

K = 3 ou 5 (intermediário)

Costuma ser mais **estável** — melhor equilíbrio entre viés e variância.

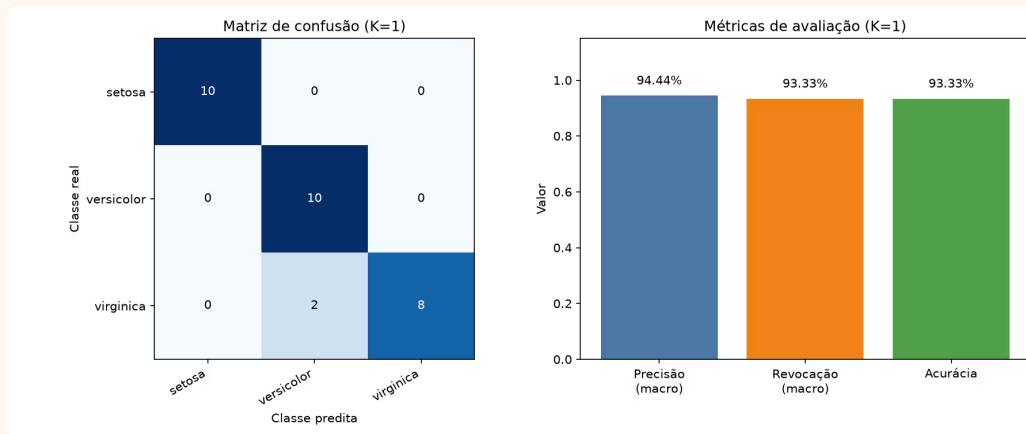
K = 7 (maior)

Pode **suavizar** a fronteira de decisão, mas às vezes piora se a fronteira for "local".

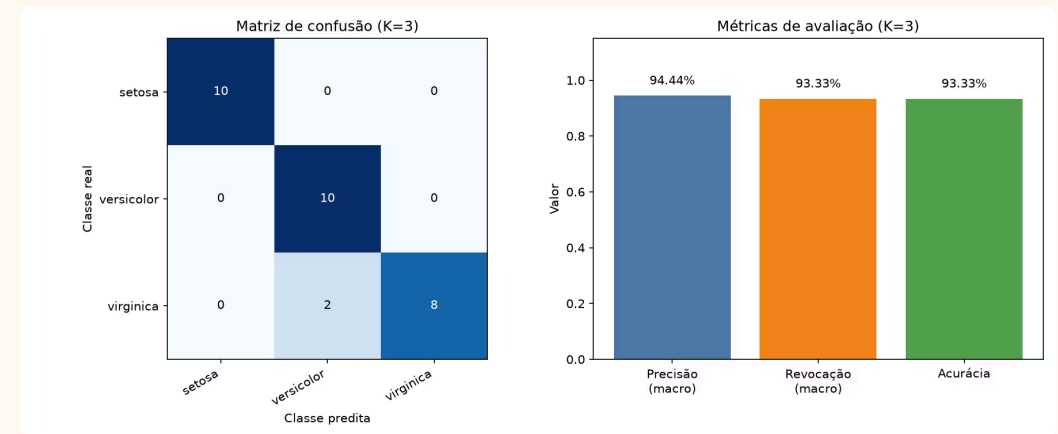
Comparação final: matrizes de confusão – Implementação Manual (Hardcore)

Diagonais mais "cheias" = melhor acerto; erros aparecem como valores *off-diagonal*.

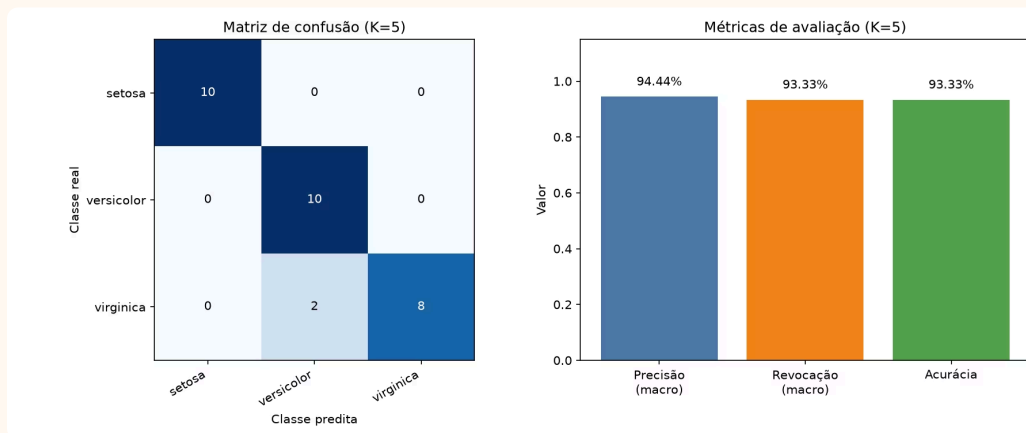
K=1



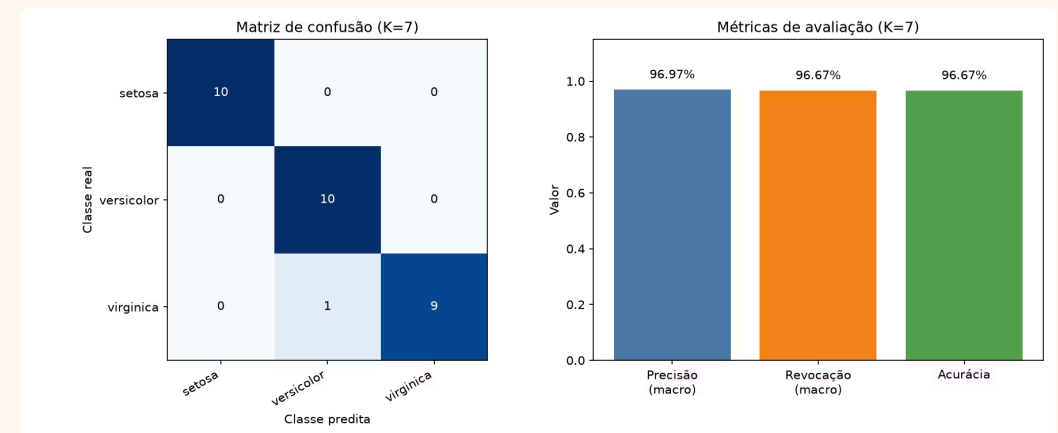
K=3



K=5



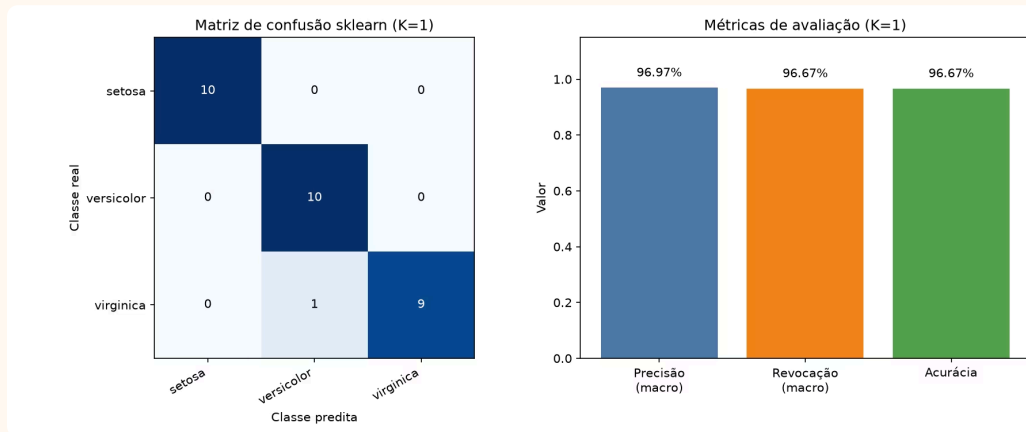
K=7



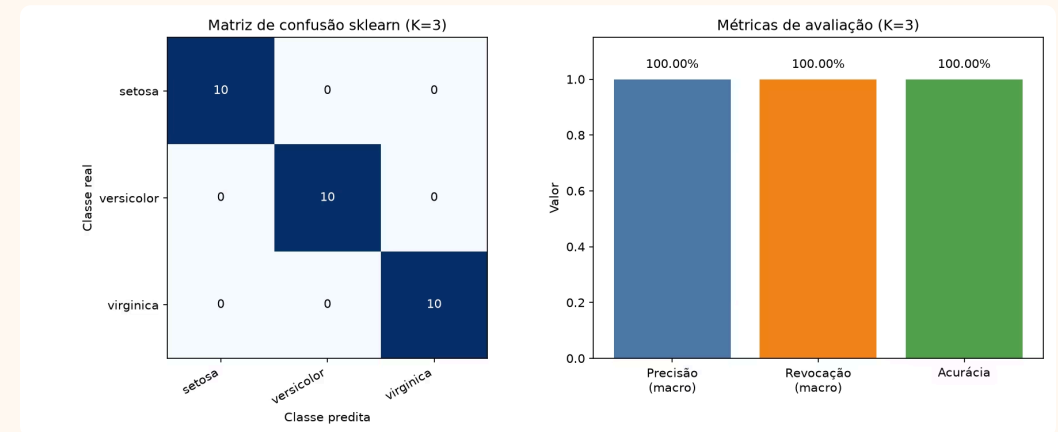
Comparação final: matrizes de confusão — Implementação Scikit-learn

Comparação direta com a versão manual sob os mesmos valores de K e mesma métrica euclidiana.

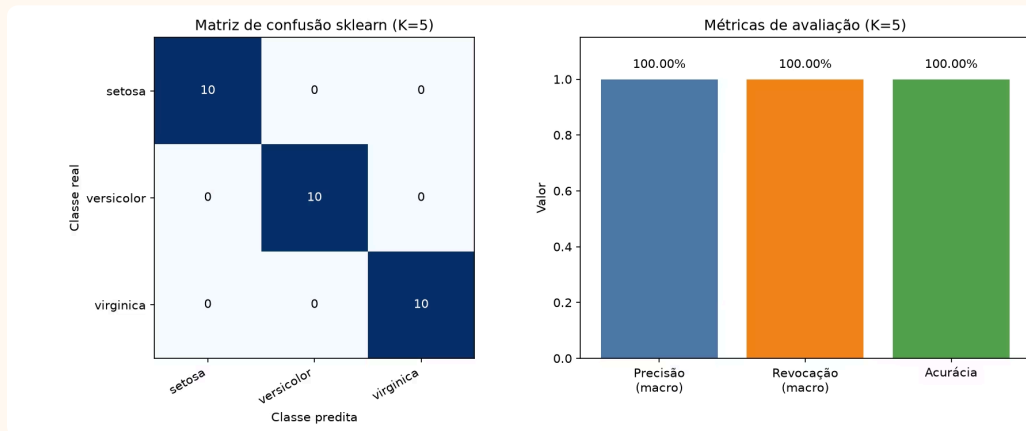
K=1



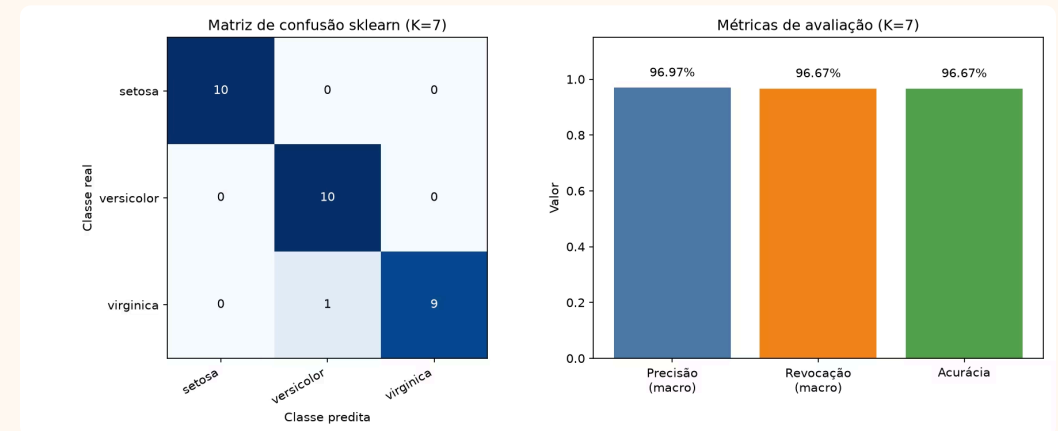
K=3



K=5



K=7



Acurácia

Mede o **acerto geral** do modelo — proporção de predições corretas sobre o total.

Precisão (macro)

Mede a **qualidade do que foi previsto** — média da precisão por classe, sem ponderação.

Revocação (macro)

Mede a **capacidade de encontrar cada classe** — média do recall por classe.

Conclusão

Principais achados

1 KNN do zero funciona

A implementação manual reproduz o comportamento esperado do algoritmo, validando a lógica de distância euclidiana e votação majoritária.

2 Sklearn reduz erros

A biblioteca facilita a implementação e reduz a chance de erro manual em etapas como divisão estratificada e cálculo de métricas.

3 K importa

A escolha do K influencia diretamente precisão e revocação — o melhor K tende a ser intermediário (normalmente 3 ou 5, a confirmar com os resultados).